

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Resumen Teórico

XML y JSON — Bases documentales y MongoDB
Práctico N.º 7

Alumno: Tomás Rodeghiero
Año: 2026

Índice

1. Datos semiestructurados: idea general	2
2. XML	2
2.1. Esquemas: DTD y XML Schema	3
2.2. Consultas y transformaciones	3
3. JSON	4
3.1. Estructura de un documento JSON	4
3.2. JSON como modelo de datos	5
4. Consultas sobre JSON	5
4.1. JsonPath en detalle	5
4.2. Ejemplos sobre la tienda de libros	6
4.3. Uso desde Java (Jayway)	6
5. Bases de datos documentales: MongoDB	6
5.1. Conceptos básicos	7
5.2. Operaciones CRUD básicas	7
5.3. Operadores de consulta	7
5.4. Aggregation framework	7
5.5. Decisiones de modelado documental	8
5.6. Validación con \$jsonSchema	8
6. XML vs JSON: comparación	9
7. Cómo se conecta con el Práctico 7	9
8. Conclusión	9

1. Datos semiestructurados: idea general

Las bases de datos clásicas exigen que cada fila tenga *exactamente* los mismos campos que el esquema de su tabla. Esto es ideal cuando los datos están bien definidos, pero genera problemas cuando hay que intercambiar datos entre aplicaciones diferentes —cada una con su propia idea de qué guardar— o cuando los datos son intrínsecamente *irregulares* (no todos los productos tienen un ISBN, no todos los clientes tienen un segundo teléfono, etc.).

Los formatos **semiestructurados** resuelven este desbalance. Un documento semiestructurado lleva “etiquetas” o “claves” que identifican cada pedazo del dato, y la estructura puede variar de un documento a otro. Los dos formatos que dominan hoy son:

- **XML**: surgió como lenguaje estándar para intercambio de datos entre sistemas. Es verboso pero rico en herramientas (esquemas, transformaciones, consultas).
- **JSON**: nació como serialización de objetos JavaScript y se volvió el formato dominante por su simpleza. Es la base de las APIs REST y de las bases de datos NoSQL documentales (MongoDB).

2. XML

XML (*eXtensible Markup Language*) fue definido por la World Wide Web Consortium (W3C) como lenguaje genérico para representar datos estructurados con marcas.

Idea. Un documento XML está hecho de elementos delimitados por marcas de apertura y cierre, anidables entre sí:

```
<supermercado>
  <articulo>
    <nro_articulo>10</nro_articulo>
    <descripcion>Harina</descripcion>
    <precio>20.8</precio>
  </articulo>
  <proveedor>
    <nro_proveedor>202</nro_proveedor>
    <nombre>Molinos Canuelas</nombre>
  </proveedor>
</supermercado>
```

Las marcas *describen* el contenido: el dato es “relativamente autodocumentado”.

Motivación.

- Comunicación de datos entre aplicaciones (bancos, procesamiento de órdenes entre empresas).
- Base del protocolo SOAP en Web Services.
- Texto plano con headers que indican el significado: fácil de debuggear y de pasar entre lenguajes.

Comparación con datos relacionales.

- *Más ineficiente en almacenamiento*: las marcas se repiten en cada elemento, mientras que en una tabla la estructura aparece una sola vez en el encabezado.
- *Mejor para intercambio*: cada documento es autodescriptivo y la estructura puede ser flexible.

2.1. Esquemas: DTD y XML Schema

Un documento XML *no* requiere asociarse a un esquema, pero los esquemas son críticos para el intercambio: permiten validar que el documento recibido es el esperado. Existen dos mecanismos:

DTD (*Document Type Definition*). La forma original, hoy obsoleta. Define qué elementos puede haber, qué atributos, qué subelementos y con qué cardinalidad. No restringe tipos de dato: todo es *string*.

```
<!ELEMENT element (subelements-specification) >
<!ATTLIST element (attributes) >
```

XML Schema. El estándar actual. Soporta:

- tipos de dato (int, string, date, decimal, etc.);
- restricciones de mínimos y máximos;
- tipos definidos por el usuario;
- claves (`xs:key`) y claves foráneas (`xs:keyref`) con `xs:selector` y `xs:field`;
- espacios de nombres (*namespaces*).

Está escrito en sintaxis XML (a diferencia de DTD), por lo que se valida con las mismas herramientas que cualquier otro XML.

Ejemplo de restricción de clave en XML Schema:

```
<xs:key name="claveCuenta">
  <xs:selector xpath="/banco/cuenta"/>
  <xs:field xpath="nro_cuenta"/>
</xs:key>

<xs:keyref name="fkCuenta" refer="claveCuenta">
  <xs:selector xpath="/banco/titular"/>
  <xs:field xpath="numero_cuenta"/>
</xs:keyref>
```

2.2. Consultas y transformaciones

Hay tres lenguajes estándares para trabajar con XML:

- **XPath:** lenguaje de expresiones tipo “path” para navegar el árbol XML. Es la base sobre la que se construyen los otros dos.
- **XQuery:** lenguaje de consulta general para XML (análogo a SQL para relacional).
- **XSLT:** lenguaje para transformar XML en otro XML, o en HTML.

XPath: navegación

Expresiones tipo `/banco/cuenta[saldo>400]/nro_cuenta`: bajan por el árbol seleccionando nodos que cumplen el predicado.

XQuery: FLWOR

XQuery usa la estructura **FLWOR**: *For, Let, Where, Order by, Return*. La equivalencia con SQL es:

XQuery	SQL
for	from
where	where
order by	order by
return	select
let	(sin equivalencia: variables temporales)

Ejemplo: obtener los números de cuenta con saldo > 400:

```
for $x in /banco-2/cuenta
let $nroCuenta := $x/@numero-cuenta
where $x/saldo > 400
return <numero-cuenta>{ $nroCuenta }</numero-cuenta>
```

XSLT: transformaciones

XSLT (*XML Stylesheet Language Transformations*) usa *templates* para transformar nodos XML. Cada template hace *match* sobre un patrón XPath y produce salida con `xsl:value-of`:

```
<xsl:template match="/banco-2/cliente">
  <xsl:value-of select="nombre_cliente"/>
</xsl:template>
```

Su uso típico es generar HTML a partir de XML, pero sirve para cualquier conversión XML → XML o XML → texto.

3. JSON

JSON (*JavaScript Object Notation*) es un formato liviano de intercambio de datos. Su éxito está en tres atributos: es muy simple de interpretar, es independiente del lenguaje y usa convenciones familiares para programadores de C, Java, JavaScript, Python, etc.

Por qué desplaza a XML.

- Menos verboso (no hay marcas de cierre repetidas).
- Mapeo directo a estructuras de datos nativas (objetos, arreglos).
- Es el formato natural de APIs REST.
- Es la base de almacenamiento de las bases NoSQL documentales, como MongoDB.

3.1. Estructura de un documento JSON

JSON está construido sobre dos estructuras:

- **Objeto:** conjunto desordenado de pares *nombre/valor*. Sintaxis: { "clave": valor, ... }.
- **Arreglo:** lista ordenada de valores. Sintaxis: [v1, v2, ...].

Tipos de valor. Un valor puede ser:

- un *string* entre comillas dobles;
- un *número* (entero o decimal);
- `true`, `false` o `null`;

- otro objeto;
- un arreglo.

Las estructuras se pueden anidar libremente.

Ejemplo.

```
{
  "nombre": "Carlos",
  "apellido": "Perez",
  "edad": 45,
  "direccion": {
    "calle": "San Martin",
    "ciudad": "Rio Cuarto",
    "codigoPostal": 5800
  },
  "telefonos": [
    { "tipo": "fijo", "numero": "358-45458412" },
    { "tipo": "movil", "numero": "351-468213744" }
  ]
}
```

3.2. JSON como modelo de datos

Lo interesante para una base de datos es que JSON soporta directamente:

- **Composición** (objetos embebidos, como `direccion` dentro de la persona): equivale al concepto E/R de *atributo compuesto*.
- **Multivalor** (arreglos como `telefonos`): equivale al concepto E/R de *atributo multivaluado*.
- **Heterogeneidad**: dos documentos de la misma colección pueden tener distintos campos.

Es exactamente lo contrario a la primera forma normal del modelo relacional. En vez de *romper* estos atributos en tablas separadas, los modelos documentales los mantienen *juntos* con el resto del dato.

4. Consultas sobre JSON

A diferencia de XML, no hay todavía un estándar único de consulta sobre JSON. La especificación XQuery 3.1 de W3C extiende XQuery a JSON, pero en la práctica conviven varios lenguajes específicos:

- **JsonPath**: el más usado, inspirado en XPath. Está implementado para muchos lenguajes (la cátedra usa Jayway JsonPath para Java).
- **JSONiq**: basado en XQuery.
- **JSON Query**: otro lenguaje tipo XPath.
- Cada lenguaje de programación tiene además su propio DSL (LINQ to JSON en .NET, `json_path` en Postgres, etc.).

4.1. JsonPath en detalle

JsonPath se basa en una colección de **operadores** y **funciones**:

Operador	Significado
\$	Raíz del documento.
@	Nodo actual (dentro de un predicado).
*	Comodín.
..	Recorrido en profundidad.
.<nombre>	Acceso a hijo.
['<n1>', '<n2>']	Hijos anotados entre corchetes.
[<i>, <j>]	Uno o más índices de arreglo.
[start:end]	Rango de índices.
[?(<expr>)]	Filtro: la expresión devuelve booleano.

Funciones de agregación. `min()`, `max()`, `avg()`, `length()`.

Operadores de filtro. `==`, `!=`, `<`, `<=`, `>`, `>=`, `~=` (regex), `in`, `nin`, `subsetof`, `anyof`, `noneof`, `size`, `empty`.

4.2. Ejemplos sobre la tienda de libros

Sobre el JSON clásico de `store.book[]` con `bicycle` y `expensive` en la raíz:

JsonPath	Resultado
<code>\$.store.book[*].author</code>	Autores de todos los libros.
<code>\$..author</code>	Todos los <code>author</code> del documento.
<code>\$.store.*</code>	Todo lo que la tienda almacena.
<code>\$.store..price</code>	Precios de todo lo almacenado.
<code>\$..book[2]</code>	El tercer libro.
<code>\$..book[-2]</code>	El penúltimo libro.
<code>\$..book[0,1]</code>	Los primeros dos libros.
<code>\$..book[?(@.isbn)]</code>	Libros que tienen ISBN.
<code>\$.store.book[?(@.price < 10)]</code>	Libros con precio menor a 10.
<code>\$..book[?(@.price <= \$['expensive'])]</code>	Libros considerados baratos.
<code>\$..book[?(@.author =~ /. *REES/i)]</code>	Equivalente al <code>LIKE</code> de SQL.
<code>\$..book.length()</code>	Cantidad de libros.

4.3. Uso desde Java (Jayway)

```
import com.jayway.jsonpath.JsonPath;
import java.io.File;

File jsonFile = new File("/tmp/json.txt");
String jsonExp = "$.store.book[?(@.price > 10)]";

System.out.println(JsonPath.read(jsonFile, jsonExp));
```

5. Bases de datos documentales: MongoDB

Las bases de datos NoSQL orientadas a documentos almacenan JSON (o una variante binaria como BSON, en el caso de MongoDB) como unidad de dato. **MongoDB** es el motor documental más popular y el que usa el Práctico 7.

5.1. Conceptos básicos

- **Base de datos:** contenedor lógico de colecciones.
- **Colección:** grupo de documentos relacionados (equivalente a una tabla, pero sin esquema rígido).
- **Documento:** un objeto JSON/BSON con un `_id` único. Es la unidad mínima de almacenamiento.
- **Esquema:** opcional. Se puede validar con `$jsonSchema` al crear la colección.

5.2. Operaciones CRUD básicas

```
// Insert
db.usuarios.insertOne({ nick: "moises", edad: 45, ... });
db.usuarios.insertMany([ {... }, { ... } ]);

// Read (find)
db.usuarios.find({ edad: { $gt: 40 } });
db.usuarios.find(
  { edad: { $gt: 40 } },           // filtro
  { _id: 0, nick: 1, edad: 1 }   // proyeccion
);

// Update
db.usuarios.updateOne(
  { nick: "moises" },
  { $set: { edad: 46 } }
);

// Upsert (update o insert si no existe)
db.usuarios.updateOne(
  { nick: "elias" },
  { $set: { ... } },
  { upsert: true }
);

// Delete
db.usuarios.deleteOne({ nick: "moises" });
```

5.3. Operadores de consulta

Los *filtros* se construyen con operadores tipo `$op`:

- Comparación: `$eq`, `$ne`, `$gt`, `$gte`, `$lt`, `$lte`, `$in`, `$nin`.
- Lógicos: `$and`, `$or`, `$not`, `$nor`.
- Existencia: `$exists`.
- Sobre arreglos: `$all`, `$size`, `$elemMatch`.
- Texto: expresiones regulares en literales o con `$regex`.

Proyección. El segundo parámetro de `find` dice qué campos incluir (1) o excluir (0). El `_id` viene por defecto incluido y suele excluirse explícitamente.

5.4. Aggregation framework

Cuando `find` no alcanza, MongoDB ofrece un *pipeline* de etapas. Cada etapa toma los documentos de la anterior y los transforma:

Etapa	Función
\$match	Filtra documentos (como <code>find</code>).
\$project	Selecciona y calcula campos.
\$group	Agrupar por una clave (similar a <code>GROUP BY</code>).
\$sort	Ordena.
\$limit / \$skip	Paginación.
\$unwind	Desempaqueta un arreglo en un documento por elemento.
\$lookup	Join lógico con otra colección.
\$addFields	Agrega o transforma campos.

Ejemplo: \$lookup para hacer un join lógico.

```
db.noticias.aggregate([
  {
    $lookup: {
      from: "usuarios",
      localField: "autor_nick",
      foreignField: "nick",
      as: "autor"
    }
  },
  { $unwind: "$autor" },
  { $match: { "autor.nyape": /gonzalez$/i } },
  { $sort: { fecha: 1 } }
]);
```

5.5. Decisiones de modelado documental

Embedding vs Referencing. La decisión clave al modelar en MongoDB:

- **Embedding:** anidar un documento dentro de otro. Ideal cuando el documento embebido es *parte* del padre y la consulta dominante los pide juntos. Ejemplo: dirección dentro del usuario.
- **Referencing:** guardar una referencia (clave) a otra colección. Ideal cuando las entidades tienen ciclos de vida independientes o cuando el embebido crecería sin tope. Ejemplo: comentarios de una noticia (pueden ser muchísimos y se agregan a lo largo del tiempo).

Sin foreign keys. MongoDB *no* valida referencias entre colecciones. La consistencia referencial queda del lado de la aplicación o de los validadores manuales.

Índices. Crear índices únicos sobre las claves de negocio (**nick**, **codigo**) e índices simples sobre los campos por los que se filtra o se hace **\$lookup** es indispensable para que las consultas sean eficientes a escala.

5.6. Validación con \$jsonSchema

Aunque MongoDB no exige esquema, se le puede dar uno al crear la colección:

```
db.createCollection("usuarios", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["nick", "nyape", "edad", "direccion", "telefonos"],
    properties: {
      nick: { bsonType: "string", minLength: 1 },
      edad: { bsonType: "int", minimum: 0 },
      direccion: { bsonType: "object",
```

```

        required: ["calle","altura"],
        properties: {
            calle: { bsonType: "string", minLength: 1 },
            altura: { bsonType: "int", minimum: 0 }
        },
        telefonos: { bsonType: "array", minItems: 1,
            items: { bsonType: "string", minLength: 1 }}
    }
}
});

```

Da garantías de tipos sin sacrificar la flexibilidad documental.

6. XML vs JSON: comparación

Dimensión	XML	JSON
Verbosidad	Alta (etiquetas duplicadas)	Baja
Tipos de dato	Todo string en DTD; tipados en XML Schema	Nativos: number, bool, null
Esquemas	DTD / XML Schema (maduros)	JSON Schema (más reciente)
Consultas	XPath, XQuery (estándares)	JsonPath, JSONiq (no estandarizados)
Transformaciones	XSLT	Ad-hoc (mapeo en código)
Uso típico	Documentos textuales, SOAP	APIs REST, NoSQL, frontend
Bases que lo usan	XML nativos, Oracle XML DB	MongoDB, CouchDB, Postgres JSONB

Cuándo conviene cada uno. XML sigue siendo preferible cuando hay que validar con un esquema estricto, transformar a varios formatos de salida o comunicarse con sistemas legados. JSON es la elección por defecto para APIs nuevas, almacenamiento documental y cuando el consumo es código JavaScript del lado del cliente.

7. Cómo se conecta con el Práctico 7

- **Ejercicio 1:** aplicación directa de JsonPath sobre un documento JSON (operadores de navegación y filtros).
- **Ejercicio 2:** generación programática de JSON desde Java con Gson, partiendo de datos relacionales.
- **Ejercicio 3:** modelado documental con embedding (`direccion`, `telefonos`) y referencing (`noticias`, `comentarios`).
- **Ejercicio 4:** operaciones de inserción con `upsert` para idempotencia.
- **Ejercicio 5:** consultas `find` con proyección y `aggregate` con `$lookup` para join lógico entre colecciones.

8. Conclusión

Las ideas centrales a retener:

- XML y JSON son los formatos de datos semiestructurados que dominan el intercambio entre aplicaciones. JSON está reemplazando a XML por su simpleza y por su buena integración con APIs REST y bases NoSQL.
- XML tiene un ecosistema maduro de esquemas (DTD, XML Schema) y consultas (XPath, XQuery, XSLT). JSON ofrece equivalentes más livianos: JSON Schema y JsonPath.

- Las bases documentales (MongoDB) usan JSON como modelo de datos. El modelado deja de pivotar sobre la primera forma normal y pasa a depender de decisiones de *embedding* vs *referencing* según el uso esperado.
- Las consultas en MongoDB se hacen con **find** (filtro + proyección) o con el *aggregation framework* (pipeline de etapas), que cubre desde joins lógicos con **\$lookup** hasta agregaciones grupales.